

MULESOFT – AWS INTEGRATION PROBLEM SET ·
VOLUME 2 OF 5

100 MuleSoft–AWS Integration Problems

Volume 2 — Messaging: SQS, SNS & EventBridge. 20 fully worked, real-project problems: queueing, fan-out, dead-letter handling, filtering, and event routing — the message-driven patterns that connect MuleSoft to the rest of an event-driven AWS architecture.

Problems 21–40

Anypoint Studio

Postman

MuleSoft Developer
Edition

Volume 2 — Messaging: SQS, SNS & EventBridge

Volume 1 moved files. Volume 2 moves messages. Amazon SQS, SNS, and EventBridge are how AWS-native and MuleSoft-integrated systems talk to each other asynchronously — queueing work for later, fanning a single event out to many subscribers, and routing events by pattern instead of by point-to-point connection. This volume builds those patterns the way a real project needs them: with dead-letter handling, backoff, deduplication, and filtering, not just a bare send-and-receive.

SECTION 1 Core Messaging: SQS & SNS (Problems 21–30)

#21

Sending a Message to an SQS Queue After Processing an HTTP Request

AWS

PROBLEM STATEMENT

A checkout API needs to accept an order instantly and return a fast response, while the actual fulfillment work — inventory reservation, warehouse notification — happens asynchronously in the background.

DETAILED EXPLANATION

Doing all of that fulfillment work synchronously, inside the same HTTP request, ties the caller's response time to the slowest downstream system and means any one of those systems being briefly unavailable fails the whole checkout. Publishing a message to an SQS queue instead decouples the two: the HTTP caller gets an immediate acknowledgment that the order was received, and a separate consumer processes the queued message whenever the downstream systems are ready — the standard entry point for turning a synchronous request into reliable asynchronous work.

CONNECTORS & COMPONENTS USED

CORE HTTP Listener

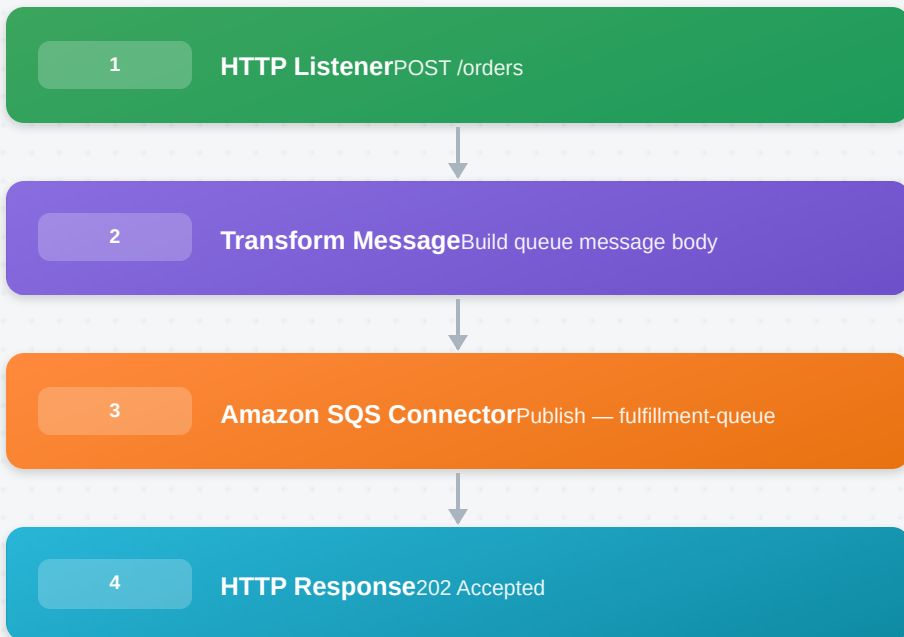
AWS Amazon SQS Connector – Publish

DW Transform Message

STEPS TO CREATE THE MULE FLOW

- 1 Expose an HTTP POST endpoint that accepts an order payload.
- 2 Shape the order into a compact message body for the queue.
- 3 Publish that message to the fulfillment SQS queue.
- 4 Return an immediate 202 Accepted response to the caller.

ACCEPT-ORDER-FLOW



MULE CONFIGURATION XML

```

<flow name="accept-order-flow">
  <http:listener path="/orders" config-ref="HTTP_Listener_config"/>
  <set-variable variableName="orderId" value="#[uuid()]" />
  <sqs:publish config-ref="SQS_Config" doc:name="Queue order for fulfillment"
    queueUrl="https://sqs.us-east-1.amazonaws.com/123456789012/fulfillment-queue">
    <sqs:body>#[%dw 2.0
output application/json
---
{
  orderId: vars.orderId,
  customerId: payload.customerId,
  items: payload.items,
  submittedAt: now()
}]]</sqs:body>
  </sqs:publish>
  <http:response statusCode="202">
    <http:body>#[{ orderId: vars.orderId, status: 'QUEUED' }]
  </http:body>
  </http:response>
</flow>

```

EXPLANATION OF THE CODE

orderId generated with uuid() before publishing

Generating the order's tracking ID in Mule, rather than waiting for a downstream system to assign one, means the caller gets a usable reference ID in the very first response, before any asynchronous processing has even started.

http:response statusCode="202" instead of 200 or 201

202 Accepted is the HTTP status specifically meant for "the request is valid and has

been accepted, but processing isn't complete yet" — the accurate status for a request that just got queued rather than fully handled.

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 Drag an `HTTP Listener` onto the canvas and set `Path` to `/orders`.
- 2 Drag a `Set Variable` named `orderId` with value `#[uuid()]`.
- 3 Search the Mule Palette for `Amazon SQS`, drag `Publish`, and set `Queue Url` and the `Body` DataWeave script.
- 4 Click the red flag on the SQS config and enter the AWS Access Key and Secret Key.
- 5 Drag an `HTTP Response` component, set `Status Code` to `202`, then save and run.

TESTING IN POSTMAN

POST `https://localhost:8081/orders`

Header	Value
Content-Type	application/json

SAMPLE REQUEST BODY

JSON

```
{
  "customerId": "CUST-5521",
  "items": [{ "sku": "WIDGET-01", "qty": 2 }]
}
```

SAMPLE RESPONSE

JSON

```
{ "orderId": "8f2a1c44-...", "status": "QUEUED" }
```

BRIEFING ALL THE STEPS IN WORDS

The HTTP Listener accepts the order and a Set Variable generates its tracking ID, the SQS Connector's Publish operation drops a compact message onto the fulfillment queue, and an immediate 202 response hands the caller their order ID before any actual fulfillment work has happened — decoupling a fast, reliable checkout response from slower downstream processing.

#22

Polling Messages from an SQS Queue and Processing Them in Mule

AWS

PROBLEM STATEMENT

The fulfillment queue from Problem #21 needs a consumer: something continuously watching the queue, picking up each order message as it arrives, and reserving inventory for it.

DETAILED EXPLANATION

Unlike S3, SQS does have a native Mule trigger — the connector's On New Message source long-polls the queue directly, waking the flow the moment a message is available rather than needing a Scheduler and a watermark. Long polling in particular matters here: it holds the connection open waiting for a message rather than immediately returning empty, which cuts both the number of API calls made against an empty queue and the latency between a message arriving and being picked up.

CONNECTORS & COMPONENTS USED

AWS

Amazon SQS Connector – On New Message

DW

Transform Message

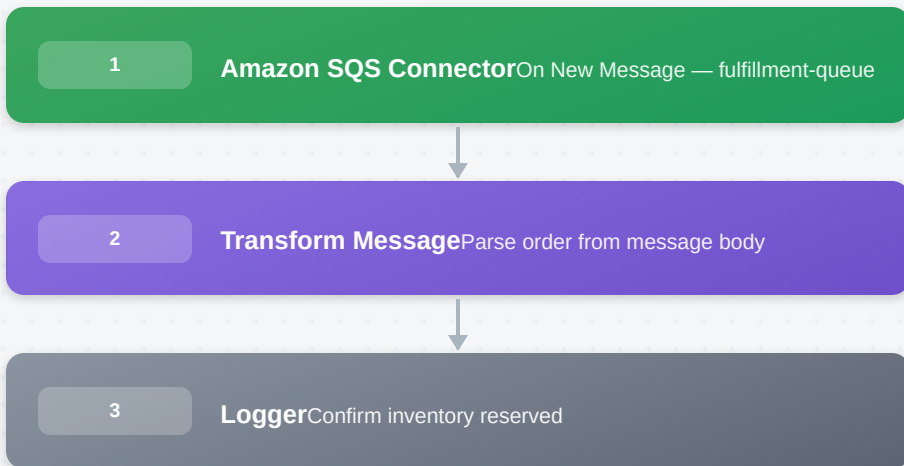
CORE

Logger

STEPS TO CREATE THE MULE FLOW

- 1 Configure the SQS connector's On New Message source against the fulfillment queue.
- 2 Parse the incoming message body back into a structured order.
- 3 Reserve inventory for the order and log the outcome.

PROCESS-ORDER-QUEUE-FLOW



MULE CONFIGURATION XML

```
<flow name="process-order-queue-flow">
  <sqs:listener config-ref="SQS_Config" doc:name="Watch fulfillment
queue"
    queueUrl="https://sqs.us-east-
1.amazonaws.com/123456789012/fulfillment-queue"
    waitTimeSeconds="20" maxMessages="10"/>
  <ee:transform doc:name="Parse order">
    <ee:message>
      <ee:set-payload>#[%dw 2.0
output application/java
---
read(payload, "application/json")]</ee:set-payload>
    </ee:message>
  </ee:transform>
  <logger level="INFO" message="#['Reserving inventory for order '
++ payload.orderId]"/>
</flow>
```

EXPLANATION OF THE CODE

`waitTimeSeconds="20"`

Setting a 20-second long-poll wait is close to SQS's own maximum of 20 seconds, which minimizes the number of empty-response API calls made while the queue has nothing new to offer.

`maxMessages="10"`

Pulling up to 10 messages per poll, rather than one at a time, lets a single polling cycle handle a burst of orders efficiently instead of round-tripping to SQS for every individual message.

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 Search the Mule Palette for `Amazon SQS` and drag `On New Message` as the flow's source.
- 2 Set `Queue Url`, `Wait Time Seconds` to `20`, and `Max Messages` to `10`.
- 3 Drag a `Transform Message` to parse the message body back into a Java object.
- 4 Drag a `Logger` to confirm processing, then save and run.

TESTING IN POSTMAN

This flow is triggered by queue arrival, not HTTP. Post an order via Problem #21's endpoint, then watch the console log for this flow picking it up.

POST

`https://localhost:8081/orders`

CONSOLE OUTPUT (THIS FLOW, MOMENTS LATER)



LOG

INFO Reserving inventory for order 8f2a1c44-...

BRIEFING ALL THE STEPS IN WORDS

The SQS Connector's On New Message source long-polls the fulfillment queue and wakes the flow as soon as an order message arrives, a transform parses the message body back into a structured order, and a logger confirms inventory was reserved — the always-on consumer side of the queue Problem #21 publishes into.

Deleting a Message from SQS After Successful Processing

AWS

PROBLEM STATEMENT

Order messages keep reappearing in the fulfillment queue after they've already been successfully processed, because nothing is telling SQS that processing actually finished.

DETAILED EXPLANATION

SQS doesn't remove a message just because a consumer received it — a received message becomes temporarily invisible to other consumers, but it's only permanently removed by an explicit Delete Message call using the receipt handle from that specific receive. If a consumer never issues that delete, the message reappears once its visibility timeout expires, on the assumption that whatever received it must have failed. Explicitly deleting after successful processing is what actually completes the SQS consumption contract.

CONNECTORS & COMPONENTS USED

AWS

Amazon SQS Connector – On New Message, Delete Message

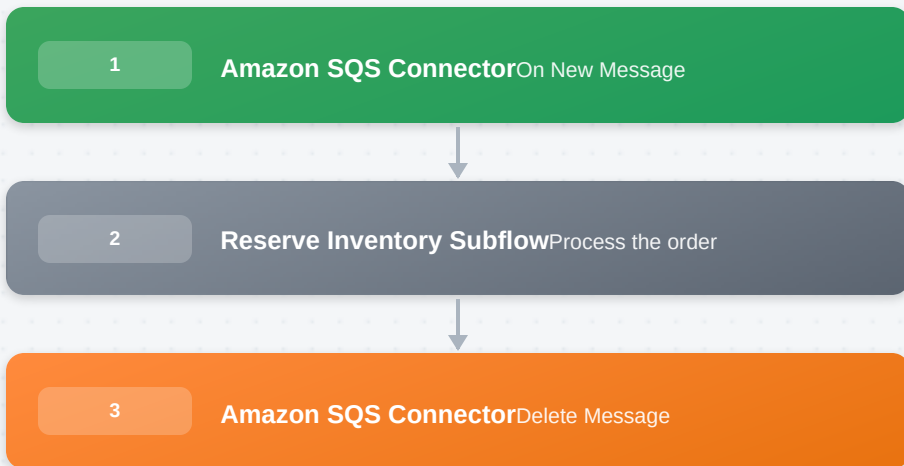
CORE

Logger

STEPS TO CREATE THE MULE FLOW

- 1 Receive a message from the queue via On New Message.
- 2 Process it (reserve inventory, as in Problem #22).
- 3 Only after processing succeeds, call Delete Message using the message's receipt handle.

PROCESS-AND-ACK-ORDER-FLOW



MULE CONFIGURATION XML

```
<flow name="process-and-ack-order-flow">
  <sqs:listener config-ref="SQS_Config" doc:name="Watch fulfillment
queue"
  queueUrl="https://sqs.us-east-
1.amazonaws.com/123456789012/fulfillment-queue"
  ackMode="MANUAL"/>
  <flow-ref name="reserve-inventory-subflow"/>
  <sqs:delete-message config-ref="SQS_Config" doc:name="Acknowledge
message"
  queueUrl="https://sqs.us-east-
1.amazonaws.com/123456789012/fulfillment-queue"
  receiptHandle="#[attributes.receiptHandle]"/>
</flow>
```

EXPLANATION OF THE CODE

ackMode="MANUAL" on the listener

Switching off automatic acknowledgment means the message stays available for redelivery unless this flow explicitly deletes it, which is exactly the safety net needed — if the flow crashes before reaching the delete

step, the message comes back instead of being silently lost.

```
receiptHandle="#[attributes.receiptHandle]"
```

The receipt handle is unique to this specific receive of the message, not to the message itself, so it has to be read from the current invocation's attributes rather than stored anywhere long-term.

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 On the SQS `On New Message` source, set `Ack Mode` to `MANUAL`.
- 2 Drag a `Flow Reference` to your inventory-reservation subflow.
- 3 Drag the SQS `Delete Message` operation and bind `Receipt Handle` to `attributes.receiptHandle`.
- 4 Save and run, then confirm in the SQS console that the queue's message count drops after processing.

TESTING IN POSTMAN

Queue-triggered. Post an order via Problem #21, then check the AWS Console's SQS queue metrics to confirm the message count returns to zero.

POST `https://localhost:8081/orders`

CONSOLE OUTPUT

LOG

```
INFO Reserving inventory for order 8f2a1c44-...  
INFO Message deleted, queue depth now 0
```

BRIEFING ALL THE STEPS IN WORDS

The SQS listener is switched to manual acknowledgment, so a Flow Reference to the inventory-reservation subflow runs first, and only once that succeeds does the SQS Connector's Delete

Message operation remove the message using its receipt handle — completing the receive-process-delete contract SQS is actually built around, instead of leaving messages to silently reappear.

#24

Handling SQS Message Visibility Timeout During Long-Running Processing

AWS

PROBLEM STATEMENT

A subset of orders take longer than the queue's default 30-second visibility timeout to process, and those orders are being picked up a second time by another consumer before the first one finishes — causing duplicate inventory reservations.

DETAILED EXPLANATION

The visibility timeout is SQS's guess at how long processing should take; once it expires, the message becomes visible to other consumers again on the assumption that the original consumer died. For a flow whose processing time is genuinely variable, the fix isn't just raising the default timeout blindly — it's actively extending the timeout for a specific in-flight message when the flow discovers it's going to take longer than expected, using Change Message Visibility to buy exactly the extra time that one message needs.

CONNECTORS & COMPONENTS USED

AWS

Amazon SQS Connector – Change Message Visibility

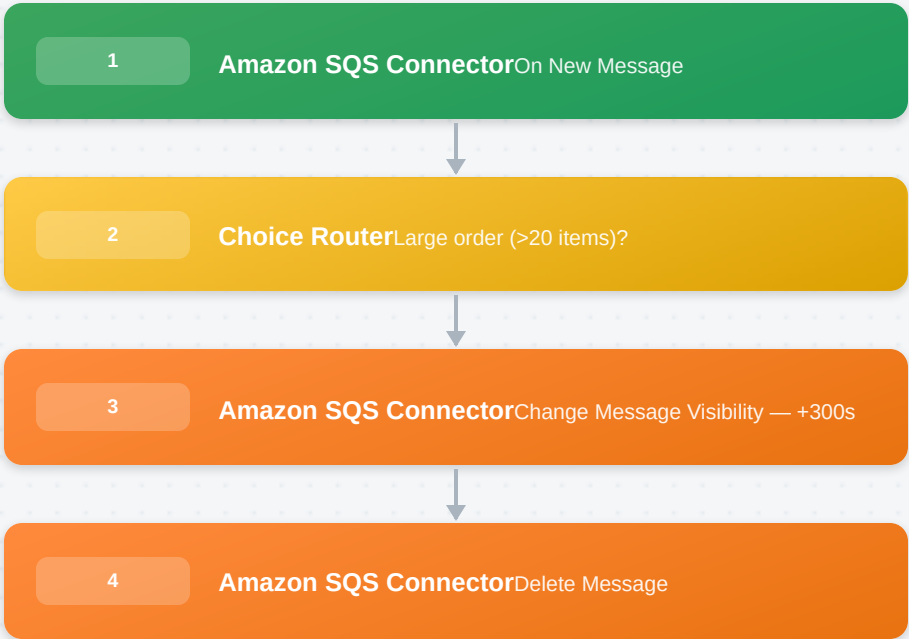
CORE

Choice Router

STEPS TO CREATE THE MULE FLOW

- 1 Receive the order message and check whether it requires extended processing (e.g. a large multi-item order).
- 2 If so, call Change Message Visibility to extend the timeout before starting the long work.
- 3 Process the order and delete the message once complete.

EXTEND-VISIBILITY-FLOW



MULE CONFIGURATION XML

```

<flow name="extend-visibility-flow">
  <sqs:listener config-ref="SQS_Config" doc:name="Watch fulfillment
queue"
    queueUrl="https://sqs.us-east-
1.amazonaws.com/123456789012/fulfillment-queue" ackMode="MANUAL"/>
  <choice>
    <when expression="#[sizeOf(payload.items) > 20]">
      <sqs:change-message-visibility config-ref="SQS_Config"
doc:name="Extend timeout for large order"
        queueUrl="https://sqs.us-east-
1.amazonaws.com/123456789012/fulfillment-queue"
        receiptHandle="#[attributes.receiptHandle]"
visibilityTimeout="300"/>
    </when>
  </choice>
  <flow-ref name="reserve-inventory-subflow"/>
  <sqs:delete-message config-ref="SQS_Config" doc:name="Acknowledge
message"
    queueUrl="https://sqs.us-east-
1.amazonaws.com/123456789012/fulfillment-queue"
    receiptHandle="#[attributes.receiptHandle]"/>
</flow>

```

EXPLANATION OF THE CODE

visibilityTimeout="300" applied conditionally

Extending the timeout only for orders actually likely to need it, rather than raising the queue's default for every message, keeps normal-sized orders redelivering quickly if their consumer genuinely does fail.

Change Message Visibility called before the long processing step, not after

The extension has to happen

while the message is still within its original visibility window — calling it too late, after the timeout has already expired, does nothing, since the message may already be back in another consumer's hands.

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 On the SQS listener, set **Ack Mode** to **MANUAL**.
- 2 Drag a **Choice** router checking the order's item count.
- 3 Inside the **when** branch, drag SQS **Change Message Visibility** and set **Visibility Timeout** to **300**.
- 4 After the choice, drag the processing subflow reference, then SQS **Delete Message**, then save and run.

TESTING IN POSTMAN

Post a large order (more than 20 items) via Problem #21's endpoint and confirm in the console that visibility is extended before processing completes.

POST

`https://localhost:8081/orders`

CONSOLE OUTPUT

LOG

```
INFO Extended visibility timeout to 300s for order 8f2a1c44-...
INFO Reserving inventory for order 8f2a1c44-...
```

BRIEFING ALL THE STEPS IN WORDS

The SQS listener runs in manual acknowledgment mode, a Choice router identifies orders large enough to risk exceeding the default timeout, and the SQS Connector's Change Message Visibility operation extends the window for exactly those messages before the long processing step runs — preventing duplicate pickups without inflating the timeout for every message on the queue.

#25

Moving a Failed SQS Message to a Dead Letter Queue

AWS

PROBLEM STATEMENT

A malformed order message keeps failing processing and cycling endlessly back onto the fulfillment queue every time its visibility timeout expires, drowning out genuinely new orders.

DETAILED EXPLANATION

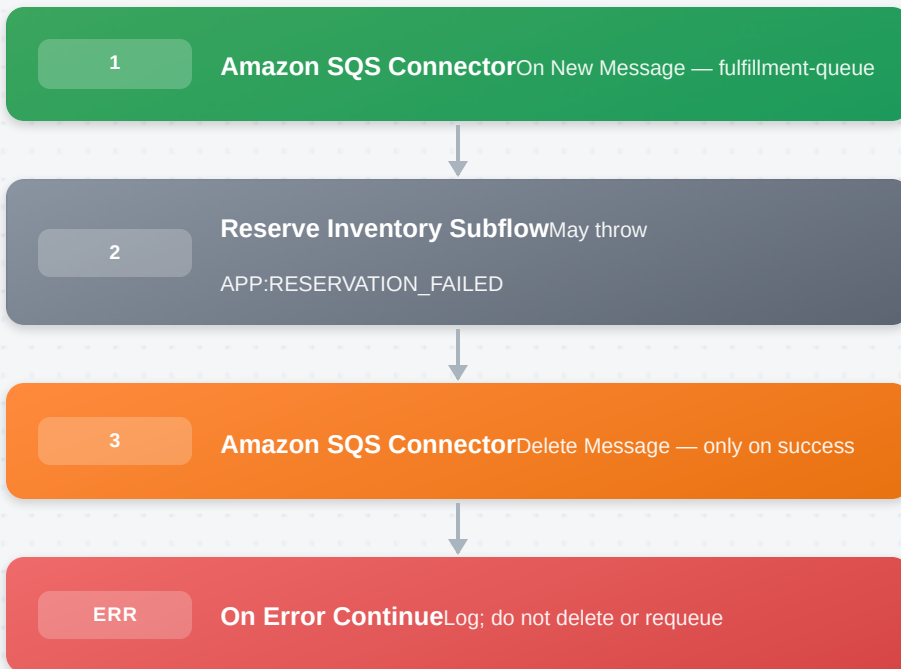
Without a Dead Letter Queue, a message that can never succeed — because it's malformed, or because it triggers a bug — simply keeps retrying forever, consuming consumer capacity and hiding among genuinely retryable failures. SQS supports configuring a redrive policy at the queue level that automatically moves a message to a separate DLQ after it's been received (and failed) a set number of times; Mule's role is simply to let that failure happen honestly — never deleting a message it couldn't actually process — so AWS's own redrive mechanism can do its job.

CONNECTORS & COMPONENTS USED

STEPS TO CREATE THE MULE FLOW

- 1 Confirm the fulfillment queue's redrive policy points at a dead-letter queue with maxReceiveCount set (configured in AWS, not Mule).
- 2 Process the message normally; if processing throws, let the error propagate without deleting the message.
- 3 Log the failure for visibility, but leave the message's fate to SQS's redrive policy.

PROCESS-ORDER-WITH-DLQ-FLOW



MULE CONFIGURATION XML

```

<flow name="process-order-with-dlq-flow">
  <sqs:listener config-ref="SQS_Config" doc:name="Watch fulfillment
queue"
    queueUrl="https://sqs.us-east-
1.amazonaws.com/123456789012/fulfillment-queue" ackMode="MANUAL"/>
  <try>
    <flow-ref name="reserve-inventory-subflow"/>
    <sqs:delete-message config-ref="SQS_Config"
doc:name="Acknowledge message"
    queueUrl="https://sqs.us-east-
1.amazonaws.com/123456789012/fulfillment-queue"
    receiptHandle="#[attributes.receiptHandle]"/>
    <error-handler>
      <on-error-continue type="APP:RESERVATION_FAILED">
        <logger level="WARN"
          message="#['Processing failed, leaving message for SQS
redrive policy: ' ++ attributes.receiptHandle]"/>
        </on-error-continue>
      </error-handler>
    </try>
  </flow>

```

EXPLANATION OF THE CODE

no `sqs:delete-message` call inside the error handler

Deliberately never deleting the message on failure is what leaves it in SQS's hands — its visibility timeout will expire, it becomes available again, and after enough failed receives the queue's own redrive policy moves it to the DLQ automatically.

`on-error-continue` rather than `on-error-propagate`

Using `continue` here lets this one message's failure be logged and the

flow move on to the next poll cycle, rather than the whole listener flow itself entering an error state over a single bad message.

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 In the AWS Console, set the fulfillment queue's **Redrive Policy** to point at a new **fulfillment-dlq** with **Maximum Receives** set to **5**.
- 2 In Anypoint Studio, wrap the processing subflow reference and delete call in a **Try** scope.
- 3 Add an **On Error Continue** handler for **APP:RESERVATION_FAILED** that only logs — no delete call inside it.
- 4 Save and run, then test with a message engineered to always fail and confirm it eventually appears in the DLQ.

TESTING IN POSTMAN

Post an order with an invalid SKU that always fails reservation, then poll the DLQ in the AWS Console after several redelivery cycles.

POST `https://localhost:8081/orders`

CONSOLE OUTPUT (AFTER 5 FAILED ATTEMPTS)



LOG

```
WARN Processing failed, leaving message for SQS redrive policy:
AQEB3...
WARN Message moved to fulfillment-dlq after 5 receives
```

BRIEFING ALL THE STEPS IN WORDS

The SQS listener stays in manual acknowledgment mode, a Try scope attempts inventory reservation and deletes the message only on success, and a failure is caught, logged, and deliberately left un-deleted so SQS's own redrive policy — not Mule — decides when a repeatedly-failing message finally moves to the dead-letter queue.

Publishing a Message to an SNS Topic When an Order Is Created

AWS

PROBLEM STATEMENT

When a new order is created, three separate teams' systems need to know — analytics, marketing, and fraud detection — and none of them should have to be individually wired into the checkout flow.

DETAILED EXPLANATION

Publishing directly to three separate SQS queues from the checkout flow would work, but it means every new consumer requires a checkout code change. SNS solves this with a publish-once, deliver-to-many model: the checkout flow publishes a single message to one topic, and any number of subscribers — SQS queues, Lambda functions, email addresses — can attach to that topic independently, entirely outside the checkout flow's knowledge or control.

CONNECTORS & COMPONENTS USED

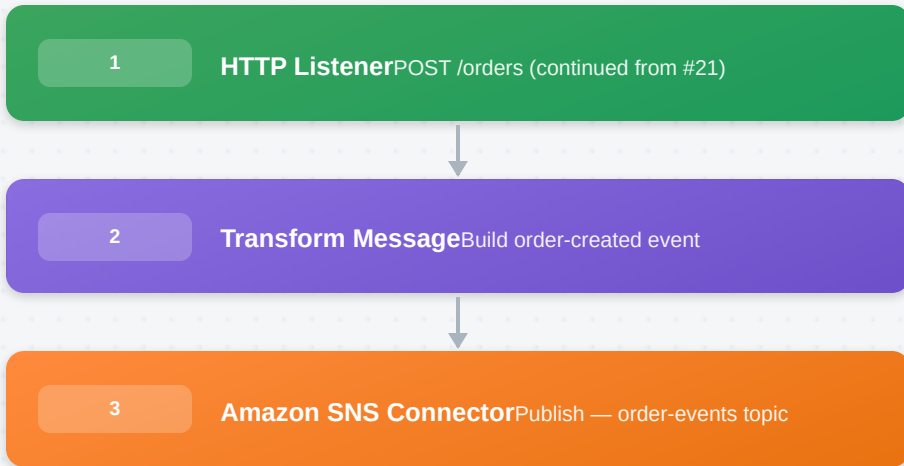
CORE HTTP Listener

AWS Amazon SNS Connector – Publish

STEPS TO CREATE THE MULE FLOW

- 1 After creating the order, build a notification payload describing the event.
- 2 Publish it to the order-events SNS topic.
- 3 Return normally — the flow has no knowledge of who's subscribed.

NOTIFY-ORDER-CREATED-FLOW



MULE CONFIGURATION XML

```
<flow name="notify-order-created-flow">
  <sns:publish config-ref="SNS_Config" doc:name="Publish order-
created event"
    topicArn="arn:aws:sns:us-east-1:123456789012:order-events"
    subject="OrderCreated">
    <sns:message>#[%dw 2.0
output application/json
---
{
  eventType: 'ORDER_CREATED',
  orderId: vars.orderId,
  customerId: payload.customerId,
  occurredAt: now()
}]]</sns:message>
  </sns:publish>
</flow>
```

EXPLANATION OF THE CODE

subject="OrderCreated"

The SNS subject line becomes the email subject for any email-protocol subscriber and is available as metadata to programmatic subscribers, so setting it explicitly gives every subscriber type a consistent, readable label for the event without them having to parse the message body first.

eventType field inside the message body

Including an explicit event type in the payload — not just relying on the topic name — means a topic that later carries more than one kind of order-related event can still be filtered and routed correctly by its subscribers.

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 Search the Mule Palette for **Amazon SNS** and drag the **Publish** operation onto the canvas after order creation.
- 2 Set **Topic Arn** to the order-events topic and **Subject** to **OrderCreated**.
- 3 Write the DataWeave **Message** body describing the event, then save and run.

TESTING IN POSTMAN

POST

`https://localhost:8081/orders`

SAMPLE RESPONSE



JSON

```
{ "orderId": "8f2a1c44-...", "status": "QUEUED" }
```

SNS PUBLISH RESULT (VISIBLE IN AWS CONSOLE MESSAGE METRICS)



JSON

```
{ "MessageId": "5a4b3c2d-...", "topicArn": "arn:aws:sns:us-east-1:123456789012:order-events" }
```

BRIEFING ALL THE STEPS IN WORDS

Once an order is created, a transform builds a small event payload describing what happened, and the SNS Connector's Publish operation sends it to a single order-events topic — letting any number of independent teams subscribe their own systems to that topic without ever touching or even being known to the checkout flow that publishes it.

#27 Subscribing an SQS Queue to an SNS Topic (Fan-Out Pattern)

AWS

PROBLEM STATEMENT

The analytics team needs a durable, ordered record of every order-created event published in Problem #26 — but SNS itself doesn't store or queue messages, so a subscriber that's briefly offline would simply miss them.

DETAILED EXPLANATION

SNS is fire-and-forget by design: it delivers to whatever's subscribed at the moment of publish, with no built-in retry queue of its own for a subscriber that's temporarily down. The standard fix is the fan-out pattern — subscribing an SQS queue to the SNS topic, so SNS's push delivery lands each message durably in a queue, and the analytics consumer can then poll that queue on its own schedule, picking up anything it missed while it was offline. This is set up once, in AWS, but Mule is what both publishes the original event and consumes from the resulting queue.

CONNECTORS & COMPONENTS USED

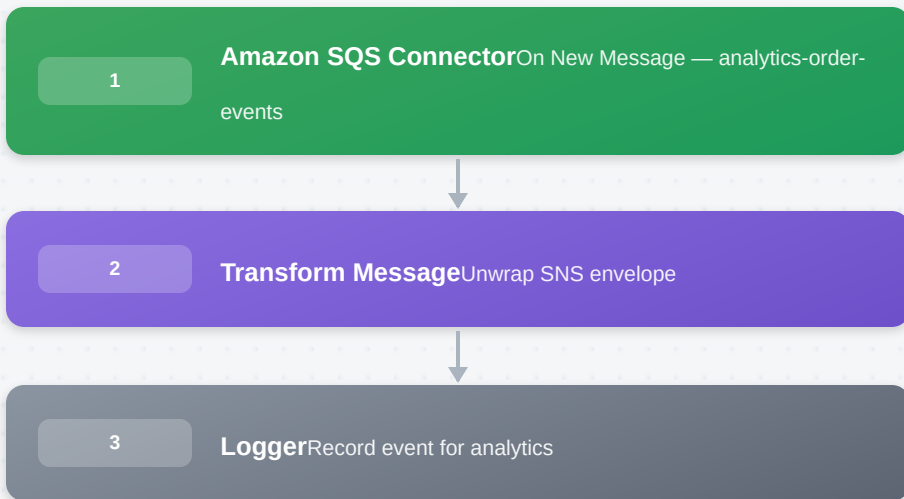
AWS Amazon SQS Connector – On New Message

DW Transform Message

STEPS TO CREATE THE MULE FLOW

- 1 In AWS, create an analytics-order-events queue and subscribe it to the order-events SNS topic.
- 2 In Mule, listen on that queue.
- 3 Unwrap SNS's envelope format to get back to the original event payload.

CONSUME-ANALYTICS-EVENTS-FLOW



MULE CONFIGURATION XML

```
<flow name="consume-analytics-events-flow">
  <sqs:listener config-ref="SQS_Config" doc:name="Watch analytics
queue"
  queueUrl="https://sqs.us-east-
1.amazonaws.com/123456789012/analytics-order-events"/>
  <ee:transform doc:name="Unwrap SNS envelope">
    <ee:message>
      <ee:set-payload>#[%dw 2.0
output application/json
var envelope = read(payload, "application/json")
---
read(envelope.Message, "application/json")]</ee:set-payload>
    </ee:message>
  </ee:transform>
  <logger level="INFO" message="#['Analytics recorded event: ' ++
payload.eventType]"/>
</flow>
```

EXPLANATION OF THE CODE

two nested `read()` calls unwrapping `envelope.Message`

When SNS delivers into a subscribed SQS queue, the queue message body is an SNS envelope (with fields like `Message`, `Subject`, `TopicArn`) whose `Message` field is itself the original JSON string that was published — so getting back to the real event payload takes two parse steps, one for the envelope and one for what it wraps.

no `delete-message` shown in this simplified example

In a production version of this flow, the same manual-ack-then-delete pattern from Problems #23–25 would apply here too; it's omitted from this listing to keep the SNS-unwrapping logic the clear focus of the example.

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 In the AWS Console, create the `analytics-order-events` SQS queue and subscribe it to the `order-events` SNS topic (Protocol: `Amazon SQS`).
- 2 In Anypoint Studio, drag SQS `On New Message` and set `Queue Url` to the new queue.
- 3 Drag a `Transform Message` and write the two-step unwrap of the SNS envelope.
- 4 Drag a `Logger` to confirm the unwrapped event, then save and run.

TESTING IN POSTMAN

Post an order via Problem #21/#26's flow, then watch this flow's console output once SNS fans the event out to the subscribed queue.

POST

`https://localhost:8081/orders`

CONSOLE OUTPUT



LOG

INFO Analytics recorded event: ORDER_CREATED

BRIEFING ALL THE STEPS IN WORDS

An SQS queue subscribed to the order-events SNS topic durably captures every published event even if the analytics consumer is briefly offline, the SQS Connector's On New Message source picks each one up, and a transform unwraps SNS's envelope format to recover the original event payload — the fan-out pattern that turns SNS's fire-and-forget delivery into something a consumer can reliably catch up on.

#28

Filtering SNS Notifications Using Message Attributes

AWS

PROBLEM STATEMENT

The fraud-detection queue subscribed to the order-events topic should only receive events for orders over \$1,000 — not every single order, which would overwhelm a system meant to review exceptions.

DETAILED EXPLANATION

Without filtering, every subscriber to a topic gets every message published to it, forcing each subscriber to do its own filtering after receiving messages it never needed. SNS subscription filter policies solve this at the source: a filter policy attached to the subscription itself tells SNS to only deliver messages whose message attributes match a given pattern, so the fraud system's queue never even receives the 95% of orders it would just discard anyway.

CONNECTORS & COMPONENTS USED

STEPS TO CREATE THE MULE FLOW

- 1 Publish the order-created event with an orderTotal message attribute, not just in the body.
- 2 In AWS, attach a filter policy to the fraud queue's subscription matching orderTotal > 1000.
- 3 Verify only qualifying orders arrive on the fraud queue.

PUBLISH-WITH-ATTRIBUTES-FLOW

1

Amazon SNS Connector Publish with messageAttributes

MULE CONFIGURATION XML

XML

```
<flow name="publish-with-attributes-flow">
  <sns:publish config-ref="SNS_Config" doc:name="Publish with
filterable attributes"
    topicArn="arn:aws:sns:us-east-1:123456789012:order-events">
    <sns:message>#[%dw 2.0 output application/json --- { eventType:
'ORDER_CREATED', orderId: vars.orderId, orderTotal: payload.total
}]]</sns:message>
    <sns:message-attributes>#[{
    orderTotal: { dataType: 'Number', stringValue: payload.total as
String },
    region: { dataType: 'String', stringValue: payload.shippingRegion
}
}]]</sns:message-attributes>
  </sns:publish>
</flow>
```

EXPLANATION OF THE CODE

`orderTotal` set as both a body field and a message attribute

SNS filter policies only evaluate message attributes, never the message body itself, so a value the body already contains still has to be duplicated as an explicit attribute if any subscriber needs to filter on it.

`dataType: 'Number'` on the `orderTotal` attribute

Declaring the correct SNS data type is what allows the fraud queue's filter policy to use a numeric comparison operator like `{"numeric": [ ">", 1000 ]}` — a policy comparing it as a string would only ever support exact-match filtering.

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 On the SNS `Publish` operation, open the `Message Attributes` field and add the DataWeave map shown above.
- 2 In the AWS Console, edit the fraud queue's SNS subscription and add a `Filter Policy` of `{"orderTotal": [{"numeric": [ ">", 1000 ]}]}`.
- 3 Save and run, then publish a small order and a large order and confirm only the large one reaches the fraud queue.

TESTING IN POSTMAN

POST

https://localhost:8081/orders

Header

Value

Content-Type

application/json

SAMPLE REQUEST BODY (QUALIFIES FOR FRAUD REVIEW)

JSON

```
{ "customerId": "CUST-5521", "total": 1450.00, "shippingRegion": "US-WEST" }
```

BRIEFING ALL THE STEPS IN WORDS

The SNS Connector's Publish operation attaches the order total as a typed message attribute alongside the usual message body, and an AWS-side filter policy on the fraud queue's subscription uses that attribute to only deliver orders over \$1,000 — keeping the filtering logic in AWS's delivery layer instead of forcing the fraud system to receive and discard everything else itself.

#29

Sending an Email Notification via SNS When a Mule Flow Fails

AWS

PROBLEM STATEMENT

When the nightly archival flow from Volume 1's Problem #20 fails outright, the platform team needs to know within minutes — by email — not discover it the next morning when someone notices the landing bucket is full.

DETAILED EXPLANATION

SNS supports email as a subscription protocol directly, alongside SQS and Lambda, which makes it a convenient way to get an operational alert to a human without standing up any separate alerting infrastructure. Wiring a global error handler to publish to an ops-alerts topic, with the platform team's email subscribed to it, means any flow in the application can raise the alarm the same way, and the team only has to manage one subscription list.

CONNECTORS & COMPONENTS USED

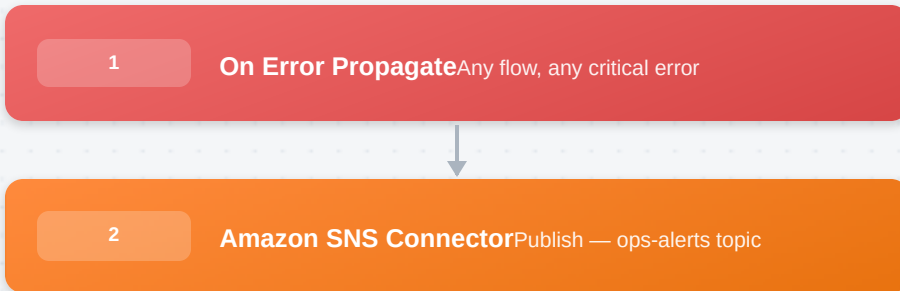
AWS Amazon SNS Connector – Publish

ERR Global Error Handler

STEPS TO CREATE THE MULE FLOW

- 1 In AWS, subscribe the platform team's email address to an ops-alerts SNS topic.
- 2 Define a global error handler that catches CRITICAL flow failures.
- 3 Publish a readable failure summary to the ops-alerts topic from that handler.

GLOBAL-ERROR-HANDLER



MULE CONFIGURATION XML

```

<error-handler name="global-ops-alert-handler">
  <on-error-propagate>
    <sns:publish config-ref="SNS_Config" doc:name="Alert platform
team"
      topicArn="arn:aws:sns:us-east-1:123456789012:ops-alerts"
      subject="#['Mule flow failure: ' ++
(error.errorType.identifier default 'UNKNOWN')]">
      <sns:message>#['Flow: ' ++ (vars.flowName default 'unknown')
++ '\nError: ' ++ error.description
++ '\nTime: ' ++ (now() as String)]</sns:message>
    </sns:publish>
  </on-error-propagate>
</error-handler>

```

EXPLANATION OF THE CODE

subject built from error.errorType.identifier

Putting the actual error type in the email subject line lets the platform team triage from their inbox without opening every alert — a **CONNECTIVITY** error and an **APP:VALIDATION** error need very different urgency.

on-error-propagate, not on-error-continue

The global handler still lets the original error propagate after sending the alert — the point of this handler is to notify a human, not to silently absorb failures that the rest of the error-handling chain (retries, DLQs) still needs to see.

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 In the AWS Console, create the **ops-alerts** SNS topic and add an **Email** subscription for the platform team's address (they'll need to confirm it once).

- 2 In Anypoint Studio's Global Elements, create an **Error Handler** named **global-ops-alert-handler** and reference it from your app's **Configuration**.
- 3 Inside it, drag the SNS **Publish** operation and build the **Subject** and **Message** from the current **error** object.
- 4 Save and run, then force a flow to fail and confirm the email arrives.

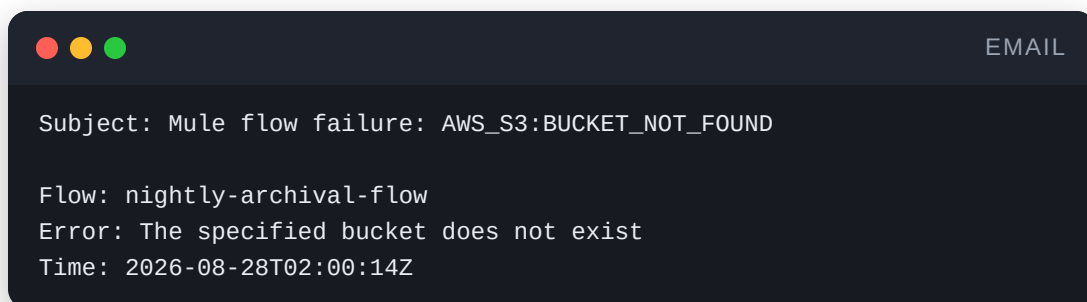
TESTING IN POSTMAN

Trigger any flow failure (e.g. an intentionally invalid S3 bucket name) and check the subscribed inbox.

POST

`https://localhost:8081/admin/force-failure-test`

SAMPLE EMAIL RECEIVED



BRIEFING ALL THE STEPS IN WORDS

A global error handler catches critical failures from any flow in the application, and the SNS Connector's Publish operation sends a readable subject and message to an ops-alerts topic that the platform team's email is subscribed to — turning a silent overnight failure into an email that arrives within seconds, using nothing more than an SNS topic and one subscription.

#30

Batch-Sending Multiple Messages to SQS in a Single Call

AWS

PROBLEM STATEMENT

A bulk order-import job needs to queue 500 orders for fulfillment, but publishing them one Mule SQS Publish call at a time takes several minutes and burns through the API request budget unnecessarily.

DETAILED EXPLANATION

SQS supports batching up to 10 messages into a single Send Message Batch API call, which cuts both the wall-clock time and the number of billed requests by roughly a factor of 10 compared to one-at-a-time publishing. The tradeoff is a small amount of extra bookkeeping: each message in the batch needs a unique ID within that batch, and the response reports success and failure per message rather than for the call as a whole, so a caller still has to check which specific messages actually made it.

CONNECTORS & COMPONENTS USED

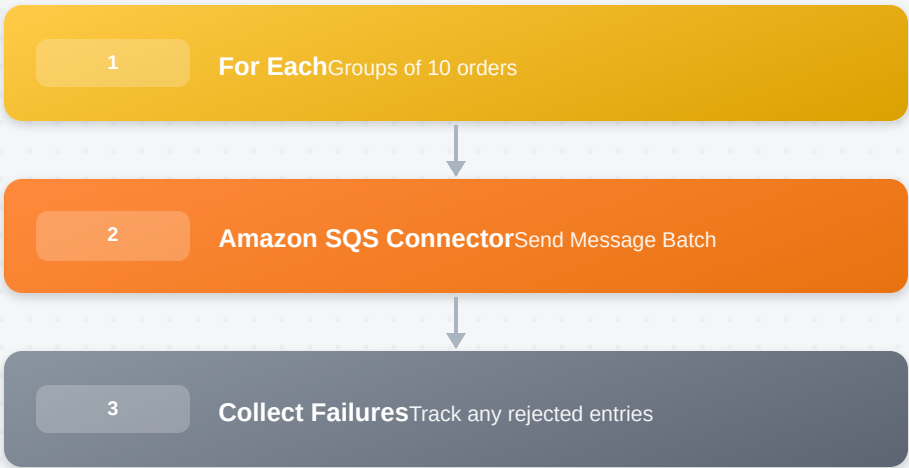
CORE For Each (batches of 10)

AWS Amazon SQS Connector – Send Message Batch

STEPS TO CREATE THE MULE FLOW

- 1 Split the 500 orders into groups of 10.
- 2 For each group, call Send Message Batch with a unique batch entry ID per order.
- 3 Collect any per-message failures across all batches for retry.

BULK-QUEUE-ORDERS-FLOW



MULE CONFIGURATION XML

```

<flow name="bulk-queue-orders-flow">
  <set-variable variableName="failedEntries" value="#[[]]"/>
  <foreach collection="#[payload divideBy 10]">
    <sqs:send-message-batch config-ref="SQS_Config"
doc:name="Batch-publish 10 orders"
    queueUrl="https://sqs.us-east-
1.amazonaws.com/123456789012/fulfillment-queue"
target="batchResult">
      <sqs:entries>#[payload map {
id: $.orderId,
messageBody: write($, "application/json")
}]</sqs:entries>
    </sqs:send-message-batch>
    <set-variable variableName="failedEntries"
      value="#[vars.failedEntries ++ (vars.batchResult.failed
default [])]"/>
    </foreach>
    <logger level="INFO" message="#['Batch queue complete, ' ++
(sizeOf(vars.failedEntries) as String) ++ ' entries failed']"/>
  </flow>

```

EXPLANATION OF THE CODE

payload divideBy 10

DataWeave's `divideBy` function splits the full 500-order array into an array of 10-item arrays in a single expression, matching exactly the batch size SQS's Send Message Batch operation accepts per call.

id: \$.orderId inside the entries mapping

Each entry in a batch needs an ID unique within that specific batch call so the response can report success or failure per entry; reusing the order's own ID both satisfies that requirement and makes the failure list directly actionable.

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 Drag a **Set Variable** named **failedEntries** initialized to an empty array.
- 2 Drag a **For Each** scope with **Collection** set to **#[payload divideBy 10]**.
- 3 Inside it, drag the SQS **Send Message Batch** operation and write the **Entries** DataWeave mapping.
- 4 After the batch call, append any **failed** entries to **failedEntries**, then log the total after the loop and save and run.

TESTING IN POSTMAN

POST

https://localhost:8081/orders/bulk-import

Header

Value

Content-Type

application/json

SAMPLE RESPONSE



JSON

```
{ "totalOrders": 500, "batchesSent": 50, "failedEntries": 0 }
```

BRIEFING ALL THE STEPS IN WORDS

A For Each loop splits the full order list into groups of 10 using DataWeave's `divideBy` function, the SQS Connector's `Send Message Batch` operation publishes each group in a single API call, and any per-entry failures are collected across every batch into one final list — cutting a 500-message publish from 500 API calls down to 50, while still tracking exactly which individual orders need a retry.

SECTION 2 Scaling Patterns: FIFO, EventBridge & Message-Driven Pipelines (Problems 31–40)

#31

Processing SQS Messages in Parallel Using a Scatter-Gather Pattern

AWS

PROBLEM STATEMENT

Each order message pulled from the fulfillment queue needs three independent checks — inventory availability, fraud score, and shipping eligibility — and running them one after another adds up to several seconds of unnecessary sequential latency per order.

DETAILED EXPLANATION

When three checks don't depend on each other's results, running them sequentially wastes time waiting on each one in turn even though they could all be in flight at once. Mule's Scatter-Gather router sends the same message to multiple routes simultaneously and waits for all of them to complete, turning three sequential round trips into one parallel round trip bounded by whichever check is slowest — a meaningful latency win for any per-message processing that fans out into independent sub-checks.

CONNECTORS & COMPONENTS USED

AWS

Amazon SQS Connector – On New Message

CORE

Scatter-Gather

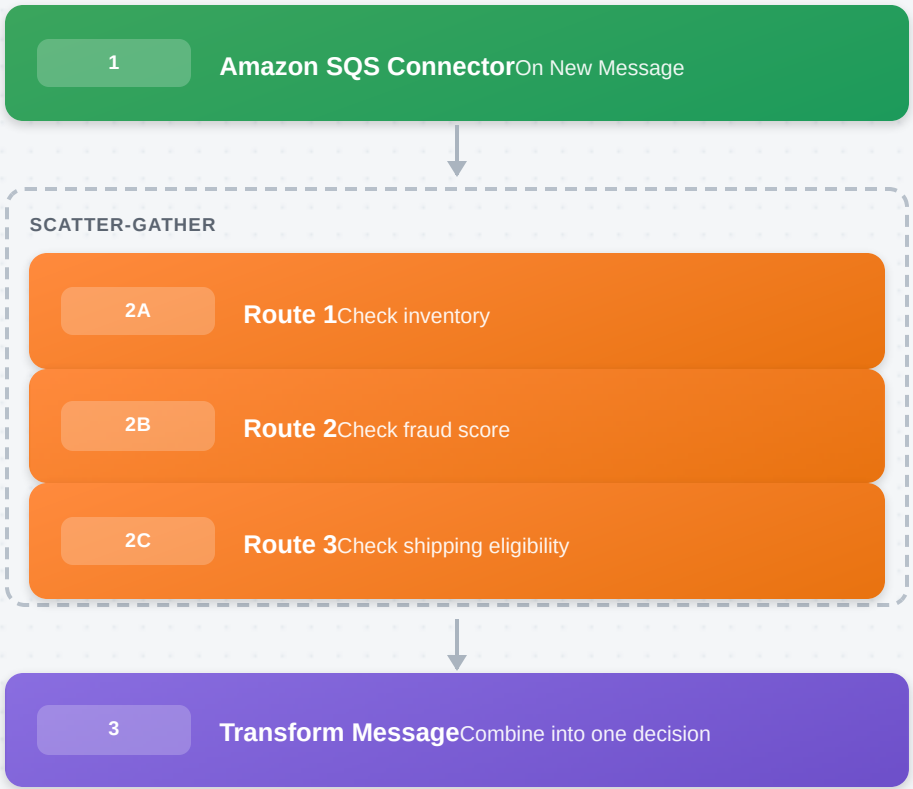
DW

Transform Message

STEPS TO CREATE THE MULE FLOW

- 1 Receive the order message from the queue.
- 2 Run inventory, fraud, and shipping checks concurrently in a Scatter-Gather router.
- 3 Combine the three results into a single eligibility decision.

PARALLEL-ORDER-CHECKS-FLOW



MULE CONFIGURATION XML

```

<flow name="parallel-order-checks-flow">
  <sqs:listener config-ref="SQS_Config" doc:name="Watch fulfillment
queue"
    queueUrl="https://sqs.us-east-
1.amazonaws.com/123456789012/fulfillment-queue"/>
  <scatter-gather doc:name="Run checks in parallel">
    <route>
      <flow-ref name="check-inventory-subflow"/>
    </route>
    <route>
      <flow-ref name="check-fraud-score-subflow"/>
    </route>
    <route>
      <flow-ref name="check-shipping-eligibility-subflow"/>
    </route>
  </scatter-gather>
  <ee:transform doc:name="Combine results">
    <ee:message>
      <ee:set-payload>#[%dw 2.0
output application/json
---
{
  inventoryOk: payload."0".payload.available,
  fraudOk: payload."1".payload.approved,
  shippingOk: payload."2".payload.eligible
}]</ee:set-payload>
    </ee:message>
  </ee:transform>
</flow>

```

EXPLANATION OF THE CODE

`payload."0", payload."1", payload."2"` after the scatter-gather

Scatter-Gather
returns a Java
Map keyed by

each route's zero-based index, with each route's own result nested inside as its **payload**, so combining the three outcomes means indexing back into that structure by route number.

three independent flow-ref calls, not three sequential ones

Because each route runs in its own route inside the scatter-gather block rather than one after another in the main flow, Mule executes all three concurrently and only proceeds once every route has returned.

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 Drag the SQS **On New Message** source onto the canvas.
- 2 Drag a **Scatter-Gather** router; it starts with two routes — add a third from its context menu.
- 3 In each route, drag a **Flow Reference** to the matching check subflow.
- 4 Drag a **Transform Message** after the scatter-gather to combine the three indexed results, then save and run.

TESTING IN POSTMAN

Queue-triggered. Post an order via Problem #21 and watch the console for the combined check result.

POST

https://localhost:8081/orders

CONSOLE OUTPUT

JSON

```
{ "inventoryOk": true, "fraudOk": true, "shippingOk": true }
```

BRIEFING ALL THE STEPS IN WORDS

The SQS listener receives each order message, a Scatter-Gather router runs inventory, fraud, and shipping checks against it simultaneously instead of one after another, and a transform combines the three indexed results into a single eligibility decision — cutting the per-order latency down to whichever single check happens to be slowest, instead of the sum of all three.

#32

Deduplicating Messages from an SQS FIFO Queue

AWS

PROBLEM STATEMENT

Payment confirmation events must be processed exactly once and in the exact order they occurred, because processing the same payment twice would double-charge a customer's account.

DETAILED EXPLANATION

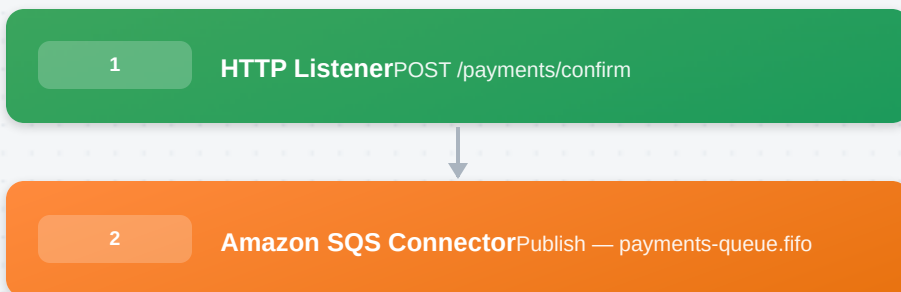
Standard SQS queues offer at-least-once delivery and no ordering guarantee, which is fine for idempotent, order-independent work but genuinely dangerous for something like payment processing. SQS FIFO queues instead guarantee exactly-once processing (within a five-minute deduplication window) and strict ordering per message group — achieved through a deduplication ID that SQS uses to silently discard a message it's already seen, and a message group ID that determines which messages must stay strictly ordered relative to each other.

CONNECTORS & COMPONENTS USED

STEPS TO CREATE THE MULE FLOW

- 1 Publish payment events to a FIFO queue with a deterministic deduplication ID.
- 2 Group messages by account ID so payments for the same account stay ordered.
- 3 Consume from the FIFO queue exactly as with a standard queue — ordering and dedup are handled by AWS.

PUBLISH-PAYMENT-FIFO-FLOW



MULE CONFIGURATION XML

```
<flow name="publish-payment-fifo-flow">
  <http:listener path="/payments/confirm" config-
ref="HTTP_Listener_config"/>
  <sqs:publish config-ref="SQS_Config" doc:name="Publish to FIFO
queue"
    queueUrl="https://sqs.us-east-
1.amazonaws.com/123456789012/payments-queue.fifo"
    messageGroupId="#[payload.accountId]"
    messageDeduplicationId="#[payload.paymentTransactionId]">
    <sqs:body>#[payload]</sqs:body>
  </sqs:publish>
</flow>
```

EXPLANATION OF THE CODE

```
messageGroupId="#[payload.accountId]"
```

Grouping by account ID means strict ordering is guaranteed only within one account's payments, which is exactly the scope that matters — two different customers' payments have no ordering relationship that needs preserving, and scoping the group this way lets SQS process different accounts' messages in parallel.

```
messageDeduplicationId="#[payload.paymentTransactionId]"
```

Using the upstream payment processor's own transaction ID as the dedup ID, rather than generating a new one in Mule, means a retried HTTP request carrying the same transaction ID is automatically deduplicated by SQS itself — even if Mule's own retry logic fires twice for the same underlying event.

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 In the AWS Console, create a FIFO queue named `payments-queue.fifo` (the `.fifo` suffix is required).
- 2 Drag an `HTTP Listener` onto the canvas and set `Path` to `/payments/confirm`.
- 3 Drag the SQS `Publish` operation, set `Queue Url` to the FIFO queue, and set `Message Group Id` and `Message Deduplication Id`.
- 4 Save and run, then send the same transaction ID twice within five minutes and confirm only one message is actually delivered downstream.

TESTING IN POSTMAN

POST

https://localhost:8081/payments/confirm

Header

Value

Content-Type

application/json

SAMPLE REQUEST BODY

JSON

```
{ "accountId": "ACC-9012", "paymentTransactionId": "TXN-773311",  
  "amount": 249.00 }
```

BRIEFING ALL THE STEPS IN WORDS

The HTTP Listener accepts a payment confirmation, and the SQS Connector's Publish operation sends it to a FIFO queue with the account ID as the message group and the payment processor's own transaction ID as the deduplication ID — letting AWS itself guarantee that payments for the same account stay strictly ordered and that a retried confirmation is never processed twice.

#33

Publishing an Event to EventBridge When a Business Transaction Completes

AWS

PROBLEM STATEMENT

When an order ships, an unknown and growing number of other AWS-native applications — not just the fixed set of teams from Problem #26 — need to react, and hardcoding every future consumer into a publish call isn't sustainable.

DETAILED EXPLANATION

SNS is a solid fit when the publisher knows roughly who its subscribers are. EventBridge is built for the opposite case: a central event bus that any number of AWS services and third-party SaaS applications can attach rules to, entirely independently, without the publisher managing subscriptions at all. Publishing a

well-structured event to EventBridge — with a clear source and detail-type — means any team can later attach a new rule to react to "order shipped" events without ever coordinating with the flow that publishes them.

CONNECTORS & COMPONENTS USED

AWS

Amazon EventBridge Connector – Put Events

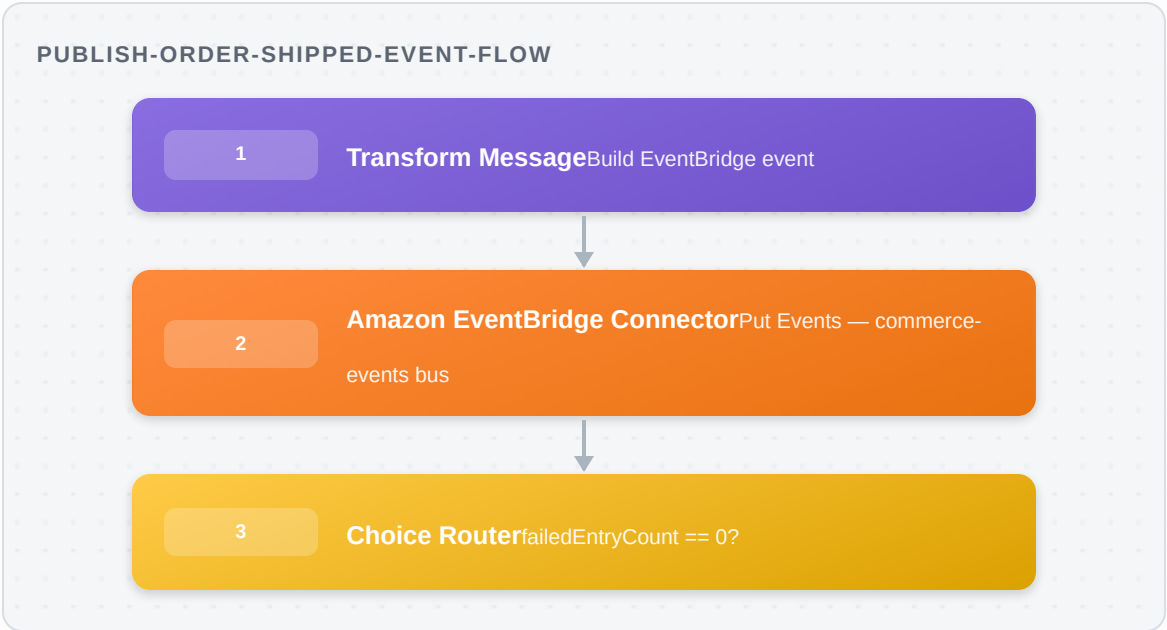
STEPS TO CREATE THE MULE FLOW

- 1

After marking an order shipped, build an EventBridge event with a clear source and detail-type.
- 2

Call Put Events against the application's custom event bus.
- 3

Confirm the event was accepted by checking the response's failedEntryCount.



MULE CONFIGURATION XML

```

<flow name="publish-order-shipped-event-flow">
  <eventbridge:put-events config-ref="EventBridge_Config"
doc:name="Publish order shipped event"
  target="putResult">
    <eventbridge:entries>#[[ {
  source: 'com.acme.commerce',
  detailType: 'OrderShipped',
  eventBusName: 'commerce-events',
  detail: write({ orderId: vars.orderId, carrier: payload.carrier,
trackingNumber: payload.trackingNumber }, "application/json")
}]]</eventbridge:entries>
  </eventbridge:put-events>
  <choice>
    <when expression="#[vars.putResult.failedEntryCount > 0]">
      <raise-error type="APP:EVENTBRIDGE_PUBLISH_FAILED"
description="EventBridge rejected the OrderShipped event"/>
    </when>
  </choice>
</flow>

```

EXPLANATION OF THE CODE

detail as a JSON string via write(...), not a raw object

EventBridge's Put Events API expects the **detail** field as a JSON-encoded string, not a nested object, because the event envelope itself is separate from whatever arbitrary shape the detail payload takes — explicitly writing it to a string is what satisfies that contract.

checking vars.putResult.failedEntryCount rather than assuming success

EventBridge's
Put Events

call can return HTTP 200 while still reporting individual entries as failed inside the response body, so a caller that only checks for an HTTP error would miss a rejected event entirely.

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 Search the Mule Palette for `Amazon EventBridge` and drag `Put Events` onto the canvas.
- 2 Write the `Entries` DataWeave array with `source`, `detailType`, `eventBusName`, and `detail`.
- 3 Set `Target Variable` to `putResult`.
- 4 Drag a `Choice` router checking `failedEntryCount`, with a `Raise Error` in the `when` branch, then save and run.

TESTING IN POSTMAN

POST

https://localhost:8081/orders/{orderId}/ship

Header

Value

Content-Type

application/json

SAMPLE REQUEST BODY



JSON

```
{ "carrier": "UPS", "trackingNumber": "1Z999AA10123456784" }
```

SAMPLE RESPONSE



JSON

```
{ "eventPublished": true, "failedEntryCount": 0 }
```

BRIEFING ALL THE STEPS IN WORDS

A transform builds an EventBridge event with a clear source, detail-type, and JSON-encoded detail payload, the EventBridge Connector's Put Events operation publishes it to the commerce-events bus, and a Choice router checks the response's failedEntryCount to catch a rejected event even when the API call itself returned success — broadcasting "order shipped" to any number of future consumers without the publisher ever needing to know who they are.

#34

Creating an EventBridge-Rule-Triggered Mule Flow for a Custom Event Pattern

AWS

PROBLEM STATEMENT

A loyalty-points service needs to react specifically to OrderShipped events for orders over \$500, ignoring every other event type and every smaller order flowing through the same commerce-events bus.

DETAILED EXPLANATION

EventBridge rules are pattern-matching filters applied at the bus level: a rule declares exactly which combination of source, detail-type, and detail field values it cares about, and only matching events are ever delivered to whatever target the rule points at. Routing that filtered subset into an SQS queue that a Mule flow consumes means the loyalty service's flow only ever sees events genuinely relevant to it — the filtering happens once, centrally, in AWS, not repeatedly inside every consumer.

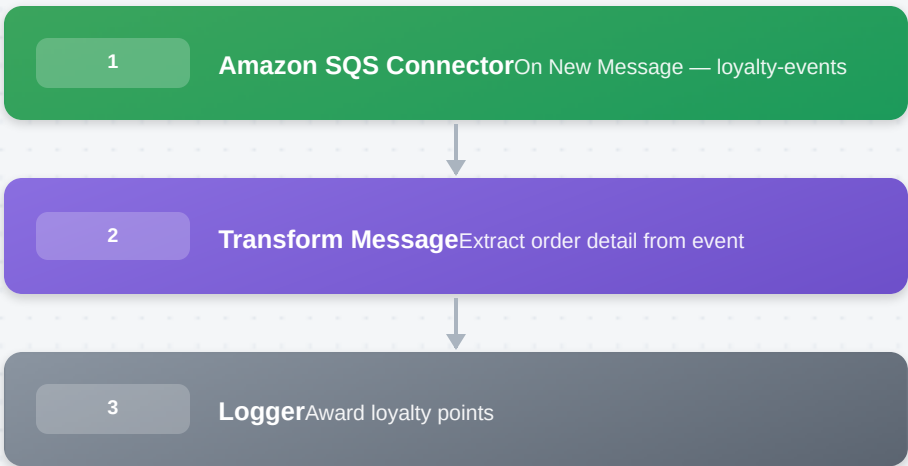
CONNECTORS & COMPONENTS USED

AWS Amazon SQS Connector – On New Message (EventBridge rule target)

STEPS TO CREATE THE MULE FLOW

- 1 In AWS, create an EventBridge rule matching source com.acme.commerce, detail-type OrderShipped, and detail.orderTotal > 500.
- 2 Set the rule's target to a dedicated loyalty-events SQS queue.
- 3 In Mule, consume from that queue exactly as with any other SQS listener.

AWARD-LOYALTY-POINTS-FLOW



MULE CONFIGURATION XML

```

<flow name="award-loyalty-points-flow">
  <sqs:listener config-ref="SQS_Config" doc:name="Watch loyalty
events queue"
    queueUrl="https://sqs.us-east-
1.amazonaws.com/123456789012/loyalty-events"/>
  <ee:transform doc:name="Extract order detail">
    <ee:message>
      <ee:set-payload>#[%dw 2.0
output application/json
var event = read(payload, "application/json")
---
read(event.detail, "application/json")]</ee:set-payload>
    </ee:message>
  </ee:transform>
  <logger level="INFO" message="#['Awarding loyalty points for
order ' ++ payload.orderId]"/>
</flow>

```

EXPLANATION OF THE CODE

`read(event.detail, "application/json")` after reading the outer event

Just like SNS's envelope in Problem #27, an EventBridge event delivered via SQS arrives as an outer JSON envelope (with source, detail-type, and

other
metadata)
whose
detail
field holds
the actual
application-
specific
payload as a
nested
JSON
string.

this flow never checks the order total itself

Because the EventBridge rule already filtered for orders over \$500 before the event ever reached this queue, the flow doesn't need to repeat that check — everything arriving here is, by construction, already relevant.

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 In the AWS Console, create an EventBridge rule with event pattern `{"source":["com.acme.commerce"],"detail-type":["OrderShipped"],"detail":{"orderTotal":{"numeric":[">",500]}}}`.
- 2 Set the rule's target to a new `loyalty-events` SQS queue.
- 3 In Anypoint Studio, drag SQS `On New Message` pointed at that queue, then a `Transform Message` to unwrap the event.
- 4 Drag a `Logger` to confirm points awarded, then save and run.

TESTING IN POSTMAN

Ship an order over \$500 via Problem #33's endpoint and confirm this flow's console output; ship one under \$500 and confirm it does NOT trigger.

POST

`https://localhost:8081/orders/{orderId}/ship`

CONSOLE OUTPUT (ORDER OVER \$500)



LOG

INFO Awarding loyalty points for order 8f2a1c44-...

BRIEFING ALL THE STEPS IN WORDS

An EventBridge rule filters the commerce-events bus down to only OrderShipped events over \$500 and delivers matches into a dedicated SQS queue, and the SQS Connector's On New Message source consumes from that queue exactly like any other — the loyalty service's flow never has to see, or filter out, a single irrelevant event.

#35

Routing Events from EventBridge to Multiple Downstream Targets

AWS

PROBLEM STATEMENT

An OrderCancelled event needs to reach both a refund-processing SQS queue and an inventory-release Lambda function, and both need to receive it independently — a failure in one shouldn't block or delay the other.

DETAILED EXPLANATION

A single EventBridge rule can have multiple targets, and EventBridge delivers to each target independently — there's no dependency or ordering between them, and a delivery failure to one target is retried on its own without affecting delivery to the others. From Mule's side, this requires no special handling at all: the publishing flow publishes exactly once, the same way as Problem #33, and it's the rule configuration in AWS, not the Mule code, that determines how many places the event ends up.

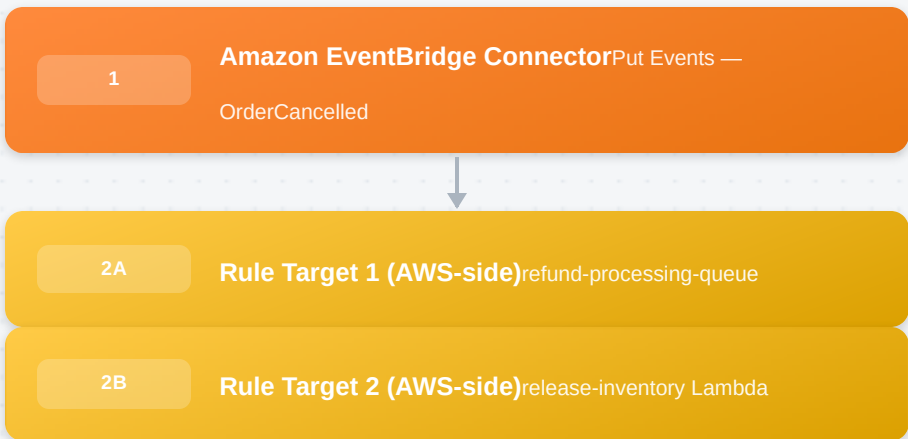
CONNECTORS & COMPONENTS USED

AWS Amazon EventBridge Connector – Put Events

STEPS TO CREATE THE MULE FLOW

- 1 Publish a single OrderCancelled event to the commerce-events bus.
- 2 In AWS, configure one rule matching OrderCancelled with two targets: the refund queue and the inventory-release Lambda.
- 3 Confirm both targets receive the event independently.

PUBLISH-ORDER-CANCELLED-FLOW



MULE CONFIGURATION XML

```
<flow name="publish-order-cancelled-flow">
  <eventbridge:put-events config-ref="EventBridge_Config"
doc:name="Publish order cancelled event">
    <eventbridge:entries>#[[{
      source: 'com.acme.commerce',
      detailType: 'OrderCancelled',
      eventBusName: 'commerce-events',
      detail: write({ orderId: vars.orderId, reason:
payload.cancelReason }, "application/json")
    }]]</eventbridge:entries>
  </eventbridge:put-events>
</flow>
```

EXPLANATION OF THE CODE

a single put-events call, identical in shape to Problem #33

The Mule side of fan-out-to-multiple-targets is deliberately unremarkable — the flow publishes one event exactly once, and it's entirely the EventBridge rule's target list, configured separately in AWS, that determines how many downstream systems ultimately receive it.

no retry or error-handling logic for the two downstream targets

Because EventBridge manages delivery and retry to each of its own targets independently,

Mule's responsibility ends at a successful Put Events call — it has no visibility into, and no responsibility for, whether the Lambda or the queue target eventually succeeds.

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 In the AWS Console, create an EventBridge rule matching `detail-type: OrderCancelled` on the commerce-events bus.
- 2 Add two targets to the rule: the `refund-processing-queue` SQS queue and the `release-inventory` Lambda function.
- 3 In Anypoint Studio, drag the EventBridge `Put Events` operation and build the OrderCancelled entry, then save and run.
- 4 Test by publishing one event and confirming both the queue and the Lambda's CloudWatch logs show it was received.

TESTING IN POSTMAN

POST

https://localhost:8081/orders/{orderId}/cancel

Header

Value

Content-Type

application/json

SAMPLE REQUEST BODY



JSON

```
{ "cancelReason": "customer_requested" }
```

BRIEFING ALL THE STEPS IN WORDS

The EventBridge Connector's Put Events operation publishes a single OrderCancelled event, and an EventBridge rule configured with two independent targets — a refund queue and an inventory-release Lambda — fans it out to both without either target's delivery depending on the other, keeping Mule's publishing logic exactly as simple as a single-target event and pushing all the fan-out complexity into AWS's own rule configuration.

#36

Replaying Failed Events from an EventBridge Archive

AWS

PROBLEM STATEMENT

A bug in the loyalty-points consumer silently dropped three hours of OrderShipped events before it was caught and fixed, and those customers' points need to be awarded retroactively without re-triggering every other consumer of the same events.

DETAILED EXPLANATION

EventBridge can archive every event that flows through a bus, and that archive supports a targeted replay: a specified time window, replayed only to a specific rule, at a controlled rate. This is exactly the tool for a "one consumer missed a window of events" recovery — a full re-publish from the source system would re-trigger every consumer, including ones that processed correctly the first time, where a scoped replay against just the loyalty rule reaches only the system that actually needs to catch up.

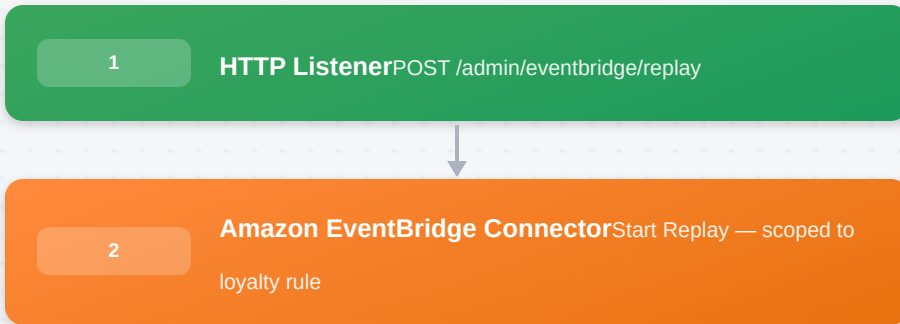
CONNECTORS & COMPONENTS USED

AWS Amazon EventBridge Connector – Start Replay

STEPS TO CREATE THE MULE FLOW

- 1 Confirm the commerce-events bus has an active archive covering the affected window.
- 2 Expose an admin endpoint that starts a replay for a given time range, scoped to the loyalty rule.
- 3 Monitor the replay until it completes.

REPLAY-LOYALTY-EVENTS-FLOW



MULE CONFIGURATION XML

```

<flow name="replay-loyalty-events-flow">
  <http:listener path="/admin/eventbridge/replay" config-
ref="HTTP_Listener_config"/>
  <eventbridge:start-replay config-ref="EventBridge_Config"
doc:name="Replay missed window to loyalty rule"
  replayName="#['loyalty-recovery-' ++ (now() as String
{format: 'yyyyMMddHHmmss'})]"
  eventSourceArn="arn:aws:events:us-east-
1:123456789012:archive/commerce-events-archive"
  eventStartTime="#[payload.startTime]"
  eventEndTime="#[payload.endTime]">
    <eventbridge:destination arn="arn:aws:events:us-east-
1:123456789012:rule/commerce-events/award-loyalty-points-rule"/>
  </eventbridge:start-replay>
</flow>

```

EXPLANATION OF THE CODE

`eventbridge:destination` pointing at the loyalty rule specifically

Scoping the replay's destination to just the loyalty rule's ARN, rather than the whole event bus, is what keeps this recovery targeted — any other rule on the same bus never sees the replayed events at all.

`replayName` built with a timestamp

EventBridge requires a unique name for every replay operation, and a timestamp-suffixed name both satisfies that uniqueness requirement and makes past replays identifiable later when reviewing what recovery actions were taken.

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 Confirm an EventBridge `Archive` is configured on the commerce-events bus (created once, in the AWS Console, capturing all events).
- 2 Drag an `HTTP Listener` onto the canvas and set `Path` to `/admin/eventbridge/replay`.
- 3 Drag the EventBridge `Start Replay` operation and set `Event Source Arn`, `Event Start Time`, `Event End Time`, and the `Destination` rule ARN.
- 4 Save and run, then trigger a replay for the affected three-hour window and confirm loyalty points are awarded for the missed orders.

TESTING IN POSTMAN

POST

`https://localhost:8081/admin/eventbridge/replay`

Header	Value
Content-Type	application/json

SAMPLE REQUEST BODY



JSON

```
{ "startTime": "2026-08-27T14:00:00Z", "endTime": "2026-08-27T17:00:00Z" }
```

SAMPLE RESPONSE



JSON

```
{ "replayName": "loyalty-recovery-20260828093012", "state": "STARTING" }
```

BRIEFING ALL THE STEPS IN WORDS

An admin endpoint accepts a time window, and the EventBridge Connector's Start Replay operation replays archived events from exactly that window to just the loyalty rule's ARN — recovering the specific consumer that missed events without re-triggering every other system that already processed them correctly the first time.

#37

Implementing Retry-with-Backoff for SQS Message-Processing Failures

AWS

PROBLEM STATEMENT

Inventory checks occasionally fail because of a transient downstream timeout, not because the order is actually invalid, and those transient failures shouldn't immediately fall through to the dead-letter queue after just one attempt.

DETAILED EXPLANATION

Relying purely on SQS's own redrive-after-N-receives mechanism (Problem #25) treats every failure identically, whether it's a permanent, unrecoverable problem or a downstream service that just needs a few seconds to recover. Wrapping the actual processing call in Mule's Until Successful scope adds in-flow retry with exponential backoff for exactly the failures worth retrying quickly, reserving the slower SQS-level redrive-to-DLQ path for failures that persist beyond what a few in-process retries can fix.

CONNECTORS & COMPONENTS USED

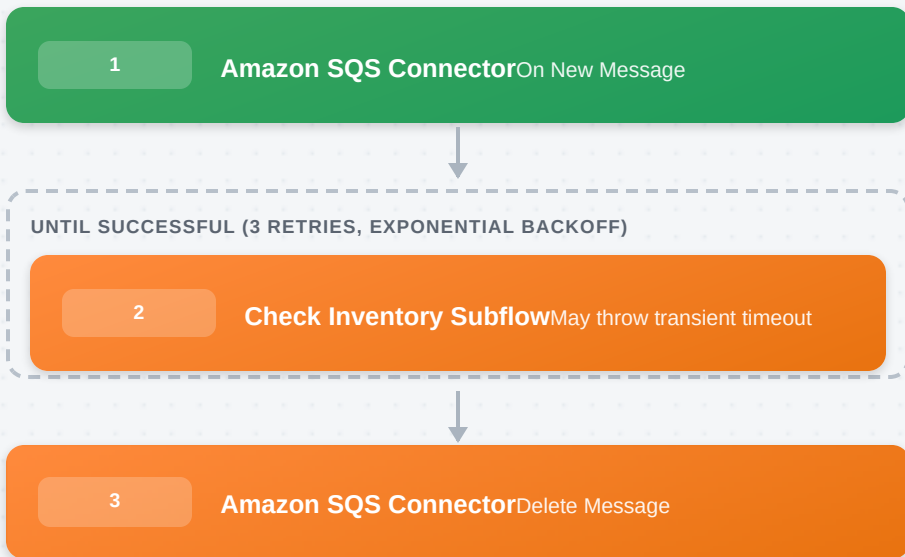
CORE Until Successful

AWS Amazon SQS Connector – Delete Message

STEPS TO CREATE THE MULE FLOW

- 1 Receive the order message.
- 2 Wrap the inventory check in an Until Successful scope with exponential backoff.
- 3 Delete the message only once the check finally succeeds, or let it fail through to SQS's own redrive after retries are exhausted.

RETRY-INVENTORY-CHECK-FLOW



MULE CONFIGURATION XML

```
<flow name="retry-inventory-check-flow">
  <sqs:listener config-ref="SQS_Config" doc:name="Watch fulfillment
queue"
  queueUrl="https://sqs.us-east-
1.amazonaws.com/123456789012/fulfillment-queue" ackMode="MANUAL"/>
  <until-successful maxRetries="3" millisBetweenRetries="2000"
  doc:name="Retry inventory check with backoff">
    <flow-ref name="check-inventory-subflow"/>
  </until-successful>
  <sqs:delete-message config-ref="SQS_Config" doc:name="Acknowledge
after successful check"
  queueUrl="https://sqs.us-east-
1.amazonaws.com/123456789012/fulfillment-queue"
  receiptHandle="#[attributes.receiptHandle]"/>
</flow>
```

EXPLANATION OF THE CODE

`maxRetries="3" millisBetweenRetries="2000"`

Until Successful in Mule 4 multiplies the base delay on each subsequent attempt, so three retries starting at 2 seconds means the inventory check gets roughly 2, 4, and 8 seconds of breathing room across three attempts before giving up — enough for a brief downstream blip to clear.

`delete-message` placed only after the `until-successful` scope

If all three retries are exhausted and the scope still fails, the flow errors out before ever reaching the delete call, so the message stays on the queue and falls back to SQS's own visibility-timeout-and-redrive mechanism from Problem #25.

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 On the SQS listener, set `Ack Mode` to `MANUAL`.
- 2 Drag an `Until Successful` scope, set `Max Retries` to `3` and `Millis Between Retries` to `2000`.
- 3 Inside it, drag a `Flow Reference` to the inventory-check subflow.
- 4 After the scope, drag SQS `Delete Message`, then save and run.

TESTING IN POSTMAN

Queue-triggered. Simulate a transient failure in the inventory subflow's downstream call and confirm retry timing in the console.

POST

`https://localhost:8081/orders`

CONSOLE OUTPUT

LOG

```
WARN  Inventory check attempt 1 failed, retrying in 2000ms
WARN  Inventory check attempt 2 failed, retrying in 4000ms
INFO  Inventory check succeeded on attempt 3
```

BRIEFING ALL THE STEPS IN WORDS

The SQS listener runs in manual acknowledgment mode, an Until Successful scope retries the inventory check up to three times with a growing delay between attempts, and the SQS Connector's Delete Message operation only fires once the check actually succeeds — catching transient failures quickly in-flow while still leaving a persistently failing message to fall back to SQS's own dead-letter handling.

#38

Correlating SNS and SQS Messages Using a Custom Correlation ID

AWS

PROBLEM STATEMENT

When an order-processing issue is reported, an engineer needs to trace a single order across three different queues and one SNS topic it passed through, but nothing currently ties those separate messages together as belonging to the same order.

DETAILED EXPLANATION

Once a business event fans out across multiple asynchronous hops — SNS to SQS, SQS to EventBridge, EventBridge back to another queue — each hop's own message ID is only meaningful for that one hop; nothing connects them unless something deliberately carries a stable identifier all the way through. Propagating the same correlation ID as a message attribute at every publish point along the chain means

an engineer can grep or filter CloudWatch Logs (or the messages themselves) by that one ID and reconstruct the entire journey of a single order.

CONNECTORS & COMPONENTS USED

AWS

SQS/SNS Connectors – correlationId attribute

CORE

Logger

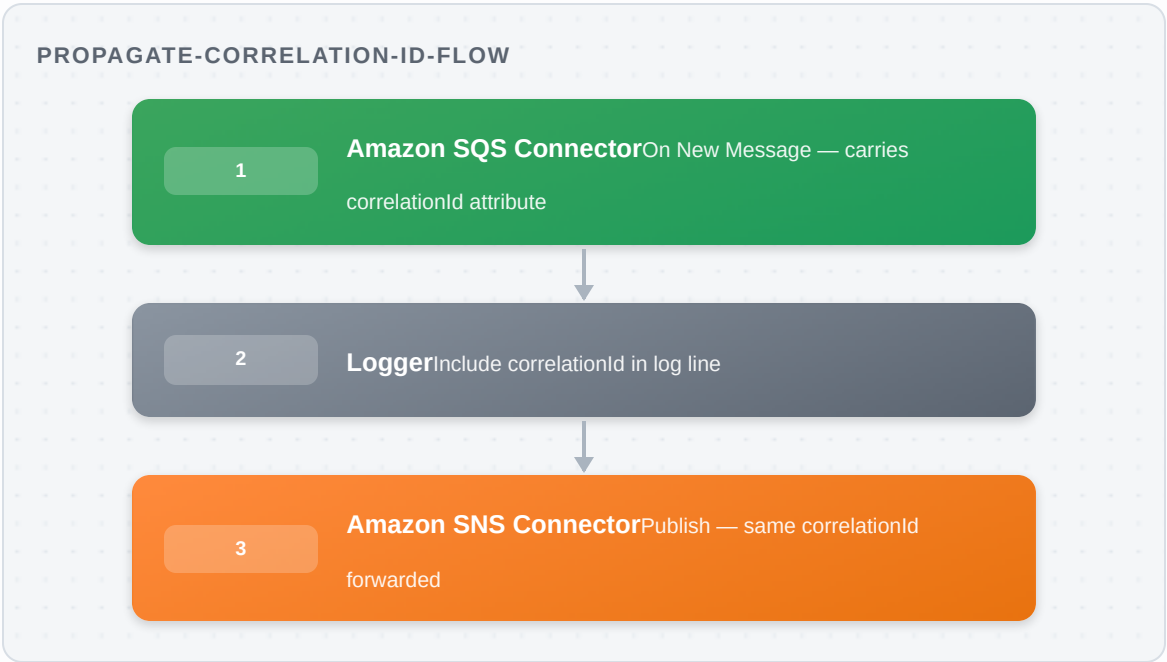
STEPS TO CREATE THE MULE FLOW

- 1

Generate (or receive) a correlation ID at the very first hop of an order's journey.
- 2

Propagate that same ID as a message attribute on every subsequent SNS publish or SQS send.
- 3

Include it in every log line so it's searchable end-to-end.



MULE CONFIGURATION XML

```

<flow name="propagate-correlation-id-flow">
  <sqs:listener config-ref="SQS_Config" doc:name="Watch fulfillment
queue"
    queueUrl="https://sqs.us-east-
1.amazonaws.com/123456789012/fulfillment-queue"/>
  <set-variable variableName="correlationId"
    value="#
[attributes.messageAttributes.correlationId.stringValue default
uuid()]" />
  <logger level="INFO" message="#[' ' ++ vars.correlationId ++ ']
Inventory reserved for order ' ++ payload.orderId]" />
  <sns:publish config-ref="SNS_Config" doc:name="Forward with same
correlation ID"
    topicArn="arn:aws:sns:us-east-1:123456789012:order-events">
    <sns:message>#[%dw 2.0 output application/json --- { eventType:
'INVENTORY_RESERVED', orderId: payload.orderId }]</sns:message>
    <sns:message-attributes>#[{ correlationId: { dataType:
'String', stringValue: vars.correlationId } }]</sns:message-
attributes>
  </sns:publish>
</flow>

```

EXPLANATION OF THE CODE

```
attributes.messageAttributes.correlationId.stringValue default uuid()
```

Reading an inbound correlation ID when one exists, and only generating a fresh one with default uuid()

when this is genuinely the first hop, is what keeps the same ID threaded through every step of a multi-hop chain instead of a new ID being minted at every stage.

```
'[' ++ vars.correlationId ++ ']' prefixing every log message
```

A consistent bracketed prefix on every log line makes the correlation ID trivially searchable in CloudWatch Logs Insights or any log aggregator, without needing structured logging just to trace one order's path.

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 Drag the SQS `On New Message` source onto the canvas.
- 2 Drag a `Set Variable` named `correlationId` that reads the inbound attribute or falls back to a new UUID.
- 3 Drag a `Logger` with the correlation ID prefixed in its message.

- 4 Drag the SNS `Publish` operation and set `Message Attributes` to forward the same `correlationId`, then save and run.

TESTING IN POSTMAN

Queue-triggered. Post an order and confirm the same correlation ID appears in this flow's log and in the forwarded SNS message attributes.

POST

`https://localhost:8081/orders`

CONSOLE OUTPUT

LOG

INFO [8f2a1c44-9b21-4e7a-...] Inventory reserved for order ORD-991

BRIEFING ALL THE STEPS IN WORDS

The flow reads an inbound correlation ID from the SQS message's attributes, falling back to a freshly generated one only if it's genuinely the first hop, logs every action with that ID as a searchable prefix, and forwards the same ID as an SNS message attribute on the next publish — giving an engineer a single ID they can search on to reconstruct an order's entire multi-hop journey across queues and topics.

#39

Building a Pub-Sub Integration Where MuleSoft Is Both Publisher and Subscriber

AWS

PROBLEM STATEMENT

The same Mule application needs to both publish `OrderShipped` events for other teams to consume, and separately subscribe to `InventoryLow` events published by the warehouse management system — acting as both a producer and a consumer within one integration layer.

DETAILED EXPLANATION

There's nothing structurally different about a Mule app being both a publisher and a subscriber — it's simply two independent flows in the same application, one that publishes and one that's triggered by an incoming message, with no coupling between them beyond sharing the same deployment. Building this cleanly means keeping those two directions in genuinely separate flows rather than one large flow doing both, so a change to the outbound publishing logic can never accidentally affect the inbound consumption logic, or vice versa.

CONNECTORS & COMPONENTS USED

AWS

Amazon SNS Connector – Publish

AWS

Amazon SQS Connector – On New Message

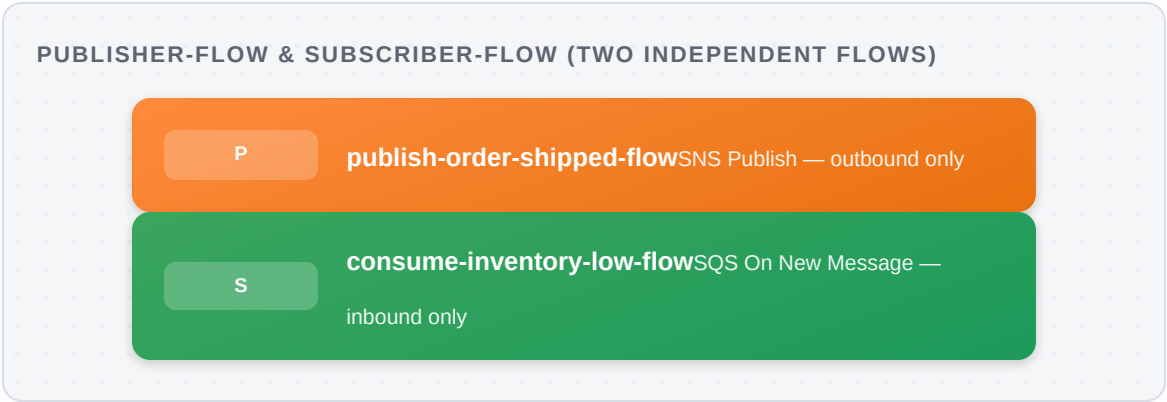
STEPS TO CREATE THE MULE FLOW

- 1

Build the outbound publisher flow for OrderShipped events (as in Problem #33).
- 2

Separately, build an inbound consumer flow subscribed to an inventory-low-alerts queue fed by the warehouse system's own InventoryLow topic.
- 3

Keep both flows entirely independent, with no shared state between them.



MULE CONFIGURATION XML

```

<flow name="publish-order-shipped-flow">
  <sns:publish config-ref="SNS_Config" doc:name="Publish
OrderShipped"
    topicArn="arn:aws:sns:us-east-1:123456789012:order-events">
    <sns:message>#[%dw 2.0 output application/json --- { eventType:
'ORDER_SHIPPED', orderId: vars.orderId }]</sns:message>
  </sns:publish>
</flow>

<flow name="consume-inventory-low-flow">
  <sqs:listener config-ref="SQS_Config" doc:name="Watch inventory-
low-alerts queue"
    queueUrl="https://sqs.us-east-
1.amazonaws.com/123456789012/inventory-low-alerts"/>
  <logger level="WARN" message="#[ 'Low inventory alert received: '
++ payload]"/>
</flow>

```

EXPLANATION OF THE CODE

two separate `<flow>` elements with no `flow-ref` between them

Nothing in this configuration connects the publisher flow to the subscriber flow — that separation is deliberate, since the two represent genuinely unrelated responsibilities that happen to be deployed in the same Mule application.

consume-inventory-low-flow watching a queue Mule doesn't publish to

This queue is fed by the warehouse

management system's own publishing, entirely outside this application; Mule's role here is purely as a consumer, mirroring the purely-publisher role of the first flow.

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 Create the `publish-order-shipped-flow` with an SNS `Publish` operation as its only real logic.
- 2 Create a second, entirely separate `consume-inventory-low-flow` starting with SQS `On New Message`.
- 3 Confirm neither flow references the other via `Flow Reference` or shared global variables.
- 4 Save and run both, then test each independently.

TESTING IN POSTMAN

Test the publisher and subscriber directions independently.

POST

`https://localhost:8081/orders/{orderId}/ship`

CONSOLE OUTPUT (SUBSCRIBER FLOW, TRIGGERED BY WAREHOUSE SYSTEM SEPARATELY)



LOG

```
WARN Low inventory alert received: {"sku":"WIDGET-01","remaining":4}
```

BRIEFING ALL THE STEPS IN WORDS

One flow publishes OrderShipped events to SNS with no knowledge of any subscriber, and a completely separate flow consumes InventoryLow alerts from an SQS queue fed by an entirely different system — both roles living in the same Mule application without being coupled to each other, proving that "publisher" and "subscriber" are just two independent flows, not two halves of one integration.

#40

Designing a Message-Driven Order-Processing Pipeline Using SQS and SNS Together

AWS

PROBLEM STATEMENT

Closing out Volume 2: assemble the individual patterns from this volume into the single, coherent order-processing pipeline a real e-commerce platform would actually run — from checkout through fulfillment through customer notification.

DETAILED EXPLANATION

This capstone combines the volume's core building blocks into one pipeline: an HTTP-triggered checkout flow queues the order onto SQS (#21) for fast acknowledgment, a queue consumer with manual acknowledgment reserves inventory and deletes the message only on success (#22, #23), a dead-letter queue catches orders that fail repeatedly (#25), and once fulfillment completes, an SNS publish (#26) fans the OrderShipped event out to every interested subscriber — analytics, loyalty, and customer notifications alike — without the pipeline needing to know who's listening.

CONNECTORS & COMPONENTS USED

CORE HTTP Listener

AWS Amazon SQS Connector – Publish, On New Message, Delete Message

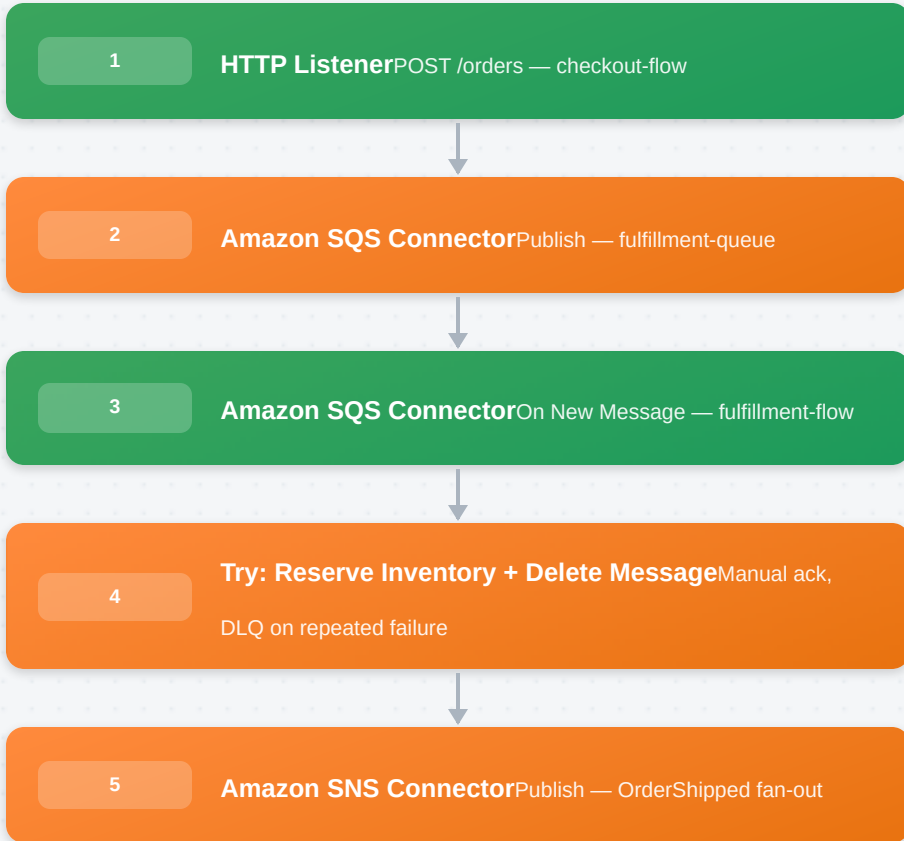
AWS Amazon SNS Connector – Publish

ERR Try / On Error Continue

STEPS TO CREATE THE MULE FLOW

- 1 Accept the order over HTTP and queue it for fulfillment, returning 202 immediately.
- 2 Consume the queue, reserve inventory inside a Try scope, and acknowledge only on success.
- 3 On success, publish an OrderShipped event to SNS for every downstream subscriber.
- 4 On repeated failure, let SQS's redrive policy route the order to the dead-letter queue for manual review.

ORDER-PROCESSING-PIPELINE (TWO FLOWS)



MULE CONFIGURATION XML

```

<flow name="checkout-flow">
  <http:listener path="/orders" config-ref="HTTP_Listener_config"/>
  <set-variable variableName="orderId" value="#[uuid()]" />
  <sqs:publish config-ref="SQS_Config" doc:name="Queue for fulfillment"
    queueUrl="https://sqs.us-east-1.amazonaws.com/123456789012/fulfillment-queue">
    <sqs:body>#[{ orderId: vars.orderId, customerId: payload.customerId, items: payload.items }]</sqs:body>
  </sqs:publish>
  <http:response statusCode="202">
    <http:body>#[{ orderId: vars.orderId, status: 'QUEUED' }]</http:body>
  </http:response>
</flow>

<flow name="fulfillment-flow">
  <sqs:listener config-ref="SQS_Config" doc:name="Watch fulfillment queue"
    queueUrl="https://sqs.us-east-1.amazonaws.com/123456789012/fulfillment-queue" ackMode="MANUAL"/>
  <try>
    <flow-ref name="reserve-inventory-subflow"/>
    <sqs:delete-message config-ref="SQS_Config" doc:name="Acknowledge"
      queueUrl="https://sqs.us-east-1.amazonaws.com/123456789012/fulfillment-queue"
      receiptHandle="#[attributes.receiptHandle]" />
    <sns:publish config-ref="SNS_Config" doc:name="Announce Order Shipped"
      topicArn="arn:aws:sns:us-east-1:123456789012:order-events">
      <sns:message>#[%dw 2.0 output application/json --- {
        eventType: 'ORDER_SHIPPED', orderId: payload.orderId }]</sns:message>
    </sns:publish>
  </error-handler>

```

```

    <on-error-continue type="APP:RESERVATION_FAILED">
      <logger level="WARN" message="Reservation failed; leaving
message for SQS redrive to DLQ"/>
    </on-error-continue>
  </error-handler>
</try>
</flow>

```

EXPLANATION OF THE CODE

`sns:publish` placed inside the try block, after `delete-message`

Announcing `OrderShipped` only after both the inventory reservation and the queue acknowledgment have succeeded guarantees the notification going out to every subscriber genuinely reflects a completed, committed fulfillment step — not a reservation that might still fail.

two independently deployable flows sharing one queue as their handoff

checkout-flow and fulfillment-flow have no direct reference to each other — the fulfillment-queue itself is

the entire interface between them, which means either side could be scaled, redeployed, or even rewritten independently as long as the queue contract stays the same.

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 Build `checkout-flow` exactly as in Problem #21: HTTP Listener, Set Variable, SQS Publish, HTTP Response.
- 2 Build `fulfillment-flow` starting with SQS `On New Message` in `MANUAL` ack mode.
- 3 Wrap the inventory-reservation subflow reference, the SQS `Delete Message` call, and the SNS `Publish` call inside a `Try` scope, in that order.
- 4 Add an `On Error Continue` handler for `APP:RESERVATION_FAILED` that only logs, leaving the message for SQS's redrive policy.
- 5 Save and run both flows, then test the full path from checkout through to the SNS announcement.

TESTING IN POSTMAN

POST

https://localhost:8081/orders

Header

Value

Content-Type

application/json

SAMPLE REQUEST BODY



JSON

```
{ "customerId": "CUST-5521", "items": [{ "sku": "WIDGET-01", "qty": 2  
}] }
```

SAMPLE RESPONSE



JSON

```
{ "orderId": "8f2a1c44-...", "status": "QUEUED" }
```

CONSOLE OUTPUT (FULFILLMENT-FLOW, MOMENTS LATER)



LOG

```
INFO  Inventory reserved for order 8f2a1c44-...  
INFO  OrderShipped event published to order-events topic
```

BRIEFING ALL THE STEPS IN WORDS

A checkout flow queues each order onto SQS and responds instantly, a separate fulfillment flow consumes that queue with manual acknowledgment, reserves inventory inside a Try scope, deletes the message only on success, and announces the completed shipment to SNS for every downstream subscriber — while a persistently failing order is deliberately left for SQS's own redrive policy to route to the dead-letter queue, closing Volume 2 by combining queueing, acknowledgment, error isolation, and fan-out into the single pipeline a real order-processing system actually runs.

End of Volume 2

Volume 2 covered the message-driven backbone of a MuleSoft–AWS integration: reliable queueing with SQS, fan-out publishing with SNS, pattern-based event routing with EventBridge, and the dead-letter, backoff, deduplication, and correlation patterns that keep a real asynchronous pipeline trustworthy under failure.

Volume 3 moves from moving messages to running compute and reading data: AWS Lambda, DynamoDB, Kinesis Data Streams, and Step Functions — the serverless and data patterns MuleSoft integrates with when a flow needs to trigger real business logic or work with structured, high-volume data — Problems 41–60.